

Live AI Hub[®] User Guide

How to design and implement an AI solution using live data from systems of record

June, 2026, Version 0.1

Copyright © 2026, Oracle and/or its affiliates

Public

Purpose statement

This document provides an overview of the Live AI Hub solution. It is intended solely to help you assess the architecture, design and implementation of AI-enabled business applications based on this solution.

Disclaimer

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle software license and service agreement, which has been executed and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

This document is for informational purposes only and is intended solely to assist you in planning for the implementation and upgrade of the product features described. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document remains at the sole discretion of Oracle. Due to the nature of the product architecture, it may not be possible to safely include all features described in this document without risking significant destabilization of the code.

Table of contents

1	Overview	5
1.1	Summary	5
1.2	Advantages of Live AI Hub	5
1.3	What this enables	5
1.4	When to use Live AI Hub	6
1.5	Example use case	6
2	Architecture	7
2.1	Logical architecture	7
2.2	Federated data access	9
2.3	Performance optimization	11
2.4	Metadata and semantics	12
2.5	Security controls	13
2.6	AI workload layer	14
2.7	Model selection	14
2.8	Deployment and locality	15
2.9	Operations, observability and governance	16
3	Design Guidance	17
3.1	Step 1 — Federate data	17
3.2	Step 2 — Augment performance	17
3.3	Step 3 — Augment metadata	18
3.4	Step 4 — Augment security	18
3.5	Step 5 — Create the AI workload	18
3.6	Step 6 — Operate the workload	19
4	Implementation Reference	20
4.1	Federate data	20
4.2	Augment performance	20
4.3	Augment metadata	20
4.4	Augment security	21
4.5	Create the AI workload	21
4.6	Operate the workload	22
5	Tutorial — Worked Example using the Retail Fashion Advisor	23
5.1	Scenario and success criteria	23
5.2	Architecture mapping	24
5.3	Build steps	25
5.3.1	Step 1 — Federate data	26
5.3.2	Step 2 — Augment performance	28
5.3.3	Step 3 — Augment metadata	29
5.3.4	Step 4 — Augment security	30

5.3.5	Step 5 — Create the AI workload	31
5.3.6	Step 6 — Operate the workload	34
6	Glossary	36

List of figures

Figure 1 Live AI Hub architecture	7
Figure 2. Federated data access	9
Figure 3. Performance optimization	11
Figure 4. Metadata and semantics	12
Figure 5. Security controls	13
Figure 6. Operations, observability and governance	16
Figure 7. Live AI Hub architecture	24
Figure 8. Fashion advisor agents and tools	25

1 Overview

1.1 Summary

Live AI Hub is a solution for bringing AI to where enterprise data lives. It works across databases, applications, data lakes and object storage, content repositories, catalogs, and data shares. It lets teams query and reason over data with minimal data movement, using a zero-ETL approach by default and selectively using caching or materialization when performance, cost, or source-system load requires it.

At a high level, the approach combines a governed data and execution layer that can access enterprise data in place and run AI capabilities, together with existing enterprise data sources that remain the systems of record.

For example, a customer might ask, “What is the current status of my order and expected delivery date, and what does your company policy say about refunds for delays?” The system can answer by combining live data from multiple systems with the relevant policy documents, without waiting for pipelines to refresh or copying the data into a new platform.

The intended result is faster time-to-value for AI initiatives, with outputs grounded in current enterprise context and protected by enterprise security controls.

1.2 Advantages of Live AI Hub

Most organizations already have the data they need to deliver AI-driven outcomes, but that data is often distributed across systems and clouds. Live AI Hub provides a way to work directly with this existing landscape, enabling teams to deliver value without requiring large-scale data movement or platform consolidation.

This approach enables several key advantages:

- Unified access across distributed data sources. Live AI Hub allows questions and workflows to span multiple systems in place, reducing fragmentation and enabling more complete, context-rich answers without building brittle integrations.
- Up-to-date insights using live data. By minimizing reliance on replicated copies and batch pipelines, the solution improves data freshness and reduces the risk of inconsistent or outdated results. This increases confidence in AI outputs, especially for operational use cases where timeliness matters.
- Reduced integration complexity and operational overhead. By limiting the need for additional pipelines and duplicate data stores, teams can simplify architecture, reduce cost, and make changes more quickly as requirements evolve.
- Built-in governance and control. The solution applies enterprise-grade security and governance controls at the point of access, including least privilege, sensitive data protection, and auditing. This ensures that AI-driven experiences operate within established policies from the start.

Live AI Hub delivers these advantages by enabling governed access and AI execution directly on live, federated data.

1.3 What this enables

Live AI Hub supports a progression of outcomes that starts with asking questions and extends to taking actions, all grounded in governed enterprise data. Once the required sources are federated, data can be analyzed and combined even when it spans different systems, models, and formats. On top of that foundation, business users and developers can interact with structured data in natural language, with SQL generated and executed against curated, governed objects.

For more advanced scenarios, agents can orchestrate multi-step workflows, gather context from multiple sources, retrieve relevant documents, and optionally trigger downstream actions through approved tools such as API calls. The same approach can also support interaction with unstructured enterprise content, allowing answers to be grounded in documents such as FAQs, policies, product literature, and knowledge articles through vector indexing and retrieval-augmented generation patterns.

Together, these capabilities make it possible to move from isolated data access to governed, AI-driven experiences that combine analysis, retrieval, reasoning, and action on live enterprise data.

1.4 When to use Live AI Hub

Live AI Hub is a strong fit when:

- A use case benefits from live access to one or more enterprise data sources. Many implementations start with only one or two sources, as long as those sources contain the data needed to answer questions or drive actions.
- Freshness matters, such as operational status, current inventory, latest orders, or up-to-date customer interactions, and waiting for ETL refresh cycles completely eliminates business value.
- AI outcomes need to be delivered quickly without starting with a large data migration or platform consolidation effort.
- Governance is a requirement from day one, including least privilege, row-level controls, sensitive data handling, and auditing for AI-mediated access.

Live AI Hub is also a practical starting point when the goal is not to replace an existing analytics strategy. Many organizations use it to deliver near-term outcomes on distributed systems, and then selectively materialize or cache data where needed for performance or cost. For some workloads, teams may later choose to evolve toward a broader lakehouse approach for large-scale curated historical analytics and standardized analytics and ML lifecycle needs.

1.5 Example use case

A representative scenario is a retail virtual shopping assistant. It illustrates why the Live AI Hub solution is needed and how it works without requiring an upfront data platform migration.

The assistant delivers personalized recommendations and answers shopping questions while a customer is actively shopping. It can pull a customer's profile and preferences, check current inventory and order or shipping status, and reference policies or product information through document retrieval, so responses stay accurate and consistent. Where appropriate, it can also take simple actions through approved tools or APIs, such as adding an item to a cart, while operating within the same security and access controls as the underlying data.

This use case specifically needs Live AI Hub because customer interactions are time-sensitive and depend on the most current operational information. Even with a lakehouse in place, batch pipelines that move data from operational systems to the lakehouse can still introduce enough delay to cause incorrect answers such as showing outdated inventory or shipment status. Live AI Hub addresses this by querying and reasoning over live data in place, with governance applied at the point of access.

At a high level, the assistant:

- Connects to multiple structured sources in place to answer questions that require cross-system context, such as customer preferences, order and shipping status, or inventory availability
- Uses agents and tools to take actions when appropriate, such as initiating an approved workflow through an API (for example, "add to cart"), while still operating under security and governance boundaries
- Uses RAG over unstructured content such as FAQs, terms and conditions, and product content, to produce grounded answers to policy and product questions.

This type of use case can improve the customer experience and help the business respond faster in day-to-day interactions. It is possible to implement Live AI Hub very quickly, without waiting for long migration projects to add new capabilities to an existing platform or complex new data pipelines to move data to where it can be processed.

2 Architecture

This section describes the logical architecture for Live AI Hub. It defines the major architectural components, the role of each layer, and the key design trade-offs addressed by the architecture, including federation, freshness, performance, metadata and semantics, security, and operations.

The architecture is designed for incremental adoption. It enables AI capabilities to be introduced into existing data environments without requiring large-scale data movement or replacement of existing systems or data platforms. Live AI Hub complements existing analytics and lakehouse strategies and can use those systems as data sources.

The [Design Guidance](#) section applies this architecture as a design workflow. It translates the architectural layers into a sequence of design decisions, validations, and controls for implementing AI on live enterprise data. The next section covering [Implementation Reference](#) then maps each step to the relevant Oracle documentation. Finally the [Tutorial](#) section includes code snippets from a worked example showing how these concepts can be used to build a working solution

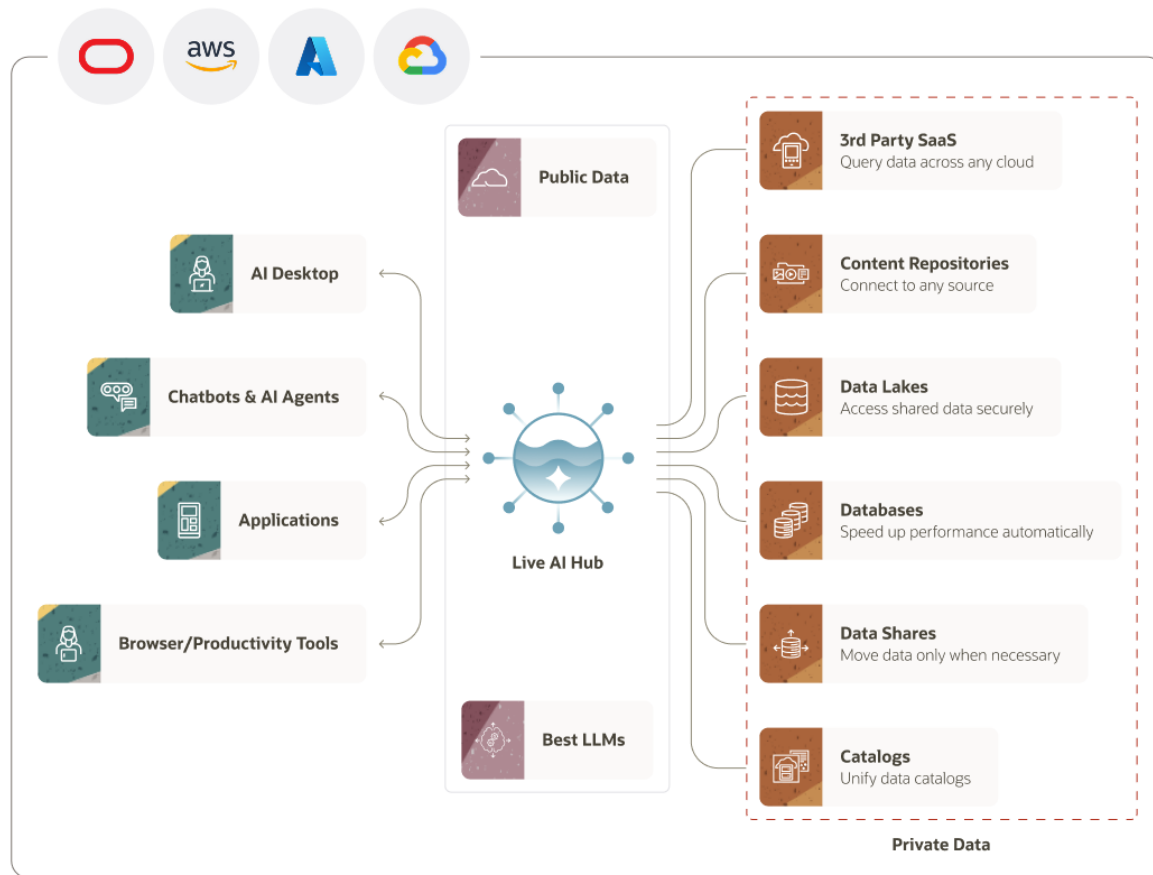


Figure 1 Live AI Hub architecture

2.1 Logical architecture

At a logical level, the architecture includes:

- A core engine (Autonomous AI Database) that provides federation, security, and AI workload execution.
- A large language model (LLM) service used for inference, and optionally embedding.
- Existing enterprise data sources, which remain systems of record.

- Optional public or third-party data accessed through APIs when needed by a use case.
- One or more client entry points, such as applications, chat interfaces, or agent frameworks, that invoke an AI workload.

Autonomous AI Database is the core of the Live AI Hub solution. It is used to:

- Connect to sources and validate access. See the section below on [federated data access](#).
- Provide access to curated data through views or external tables. See the section below on [performance optimization](#).
- Add semantic information by enriching metadata with comments and annotations. See the section below on [metadata and semantics](#).
- Enforce security controls consistently across all access paths. See the section below on [security controls](#).
- Run AI features such as NL2SQL, agents and RAG. See the section below on [AI workload layer](#).

In addition, Autonomous AI Database is used to:

- **Store agents' memory and context for simplified AI agent development and operation.** Live AI Hub can act as the persistent state layer for AI agents by storing the context they need to operate reliably across interactions, sessions, tools, and workflows. This may include conversation summaries, user or customer context, task state, previous tool results, agent decisions, workflow checkpoints, preferences, and other memory objects that help the agent continue work without rebuilding context from scratch each time. Keeping this state in the data platform simplifies agent development because memory can be queried, updated, secured, audited, and governed using the same database capabilities that protect enterprise data. It also makes agent behavior easier to operate in production: teams can inspect what context was available, manage retention and refresh rules, separate temporary session memory from longer-lived business memory, and ensure that agents only retrieve context appropriate to the user, role, and use case.
- **Store vectors, vector indexes, and run similarity or hybrid searches to enable RAG.** Live AI Hub can store embeddings and vector indexes alongside the enterprise data and metadata needed to ground AI responses. Documents, product records, knowledge articles, FAQs, policies, and other source material can be converted into vector representations and indexed for similarity search, allowing an AI workload to retrieve the most relevant content before generating an answer. Hybrid search extends this pattern by combining vector similarity with keyword, metadata, structured filters, or relational predicates, which is often important in enterprise use cases where relevance depends on both meaning and business constraints. For example, a RAG workflow may need to find semantically similar product information, but only within an approved catalog, region, customer segment, language, freshness window, or entitlement boundary. Storing vectors and indexes in the hub keeps retrieval close to governed enterprise data, reduces the need for separate vector infrastructure, and allows RAG pipelines, NL2SQL, and agents to use a common security, metadata, and audit model.

The LLM service provides model capabilities used by those features. Depending on requirements, the LLM can be served by OCI Generative AI service, another cloud hyperscaler service, or a customer-managed deployment.

2.2 Federated data access

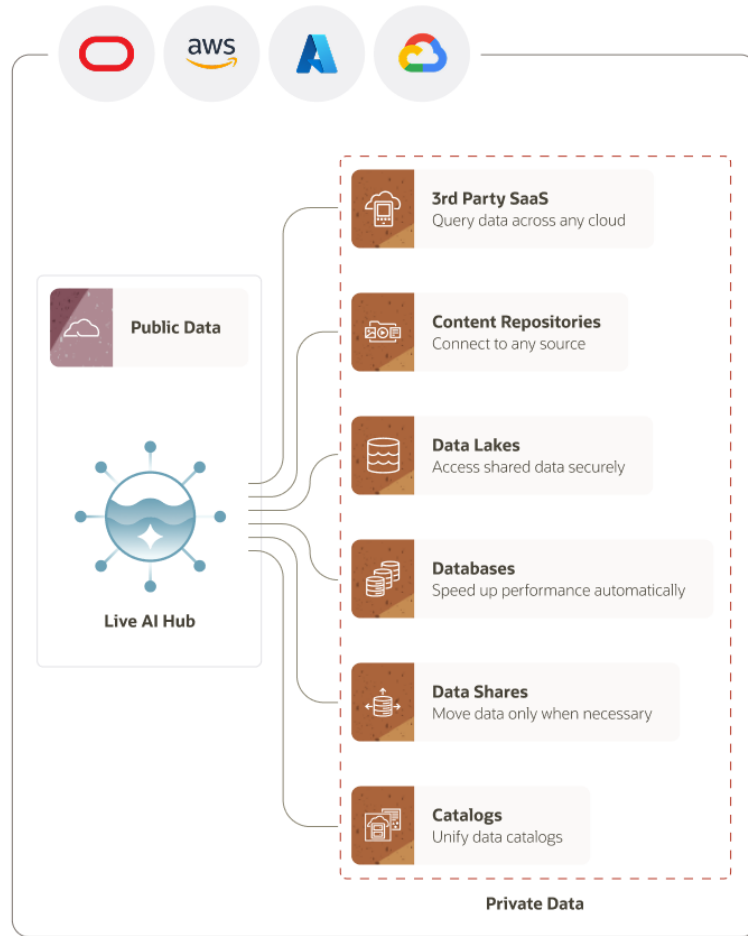


Figure 2. Federated data access

Live AI Hub is intended to work across heterogeneous enterprise sources, including:

- Operational databases.** Operational databases are often the most important source for Live AI Hub because they contain the current state of the business: customers, orders, inventory, shipments, service cases, payments, account status, and other transactional data. These systems remain the systems of record, while Live AI Hub federates approved objects from them so AI workloads can answer time-sensitive questions without waiting for batch ETL or duplicated copies to refresh. Access should normally be narrowed through curated views, roles, and security policies rather than exposing internal application schemas directly.
- Analytics systems and warehouses** Analytics systems and data warehouses provide curated historical context, governed metrics, dimensional models, precomputed KPIs, aggregations, forecasts, and model outputs. This data complements live operational data by helping AI workloads explain not only what is happening now, but also how current activity compares to trends, targets, segments, or prior periods. Reusing trusted warehouse outputs also avoids rebuilding business logic in prompts, tools, or agent code. Where appropriate, these sources can be federated directly or exposed through curated views or materialized objects.
- Data lakes and object storage.** Data lakes and object stores contain large-scale file and table data, often in open or semi-structured formats, and may be located across OCI, Azure, AWS, Google Cloud, and S3-compatible environments. These sources may include clickstream data, logs, IoT data, archives, product data, data science outputs, and curated lakehouse tables. Live AI Hub can make selected lake data available to SQL, NL2SQL, agents, and RAG workflows without requiring

the entire lake to be copied into the database. When repeated reads are expected, or when cross-cloud latency and egress costs matter, external table caching or materialized views can be added to improve performance.

- **Content repositories.** Content repositories provide the unstructured knowledge that many AI workloads need in order to produce grounded answers. Examples include policies, FAQs, product literature, manuals, contracts, knowledge articles, terms and conditions, and other enterprise documents. This content may be accessed through REST APIs, connectors, or object storage paths and is often prepared for retrieval-augmented generation using vector indexes. In a Live AI Hub design, content should be treated as governed source material: only approved repositories or document sets should be exposed, refresh expectations should be defined, and metadata should be captured so retrieval results can be traced and interpreted.
- **Catalogs and data shares.** Catalogs and data shares provide governed discovery and access to datasets that may already be managed outside the Live AI Hub environment. Catalogs help identify what data exists, who owns it, what it means, and how it should be governed. Data shares provide a controlled way to consume datasets published by another team, platform, partner, or ecosystem provider. Live AI Hub can use these mechanisms to avoid unnecessary duplication while still bringing selected datasets into an AI-ready access pattern. Before exposing cataloged or shared data to an AI workload, teams should curate the object set, confirm access rights, and add local semantics where needed.

The architecture supports multiple ways to federate and query data in place. The mechanism used depends on the source technology and how that source is exposed and governed. Common federation mechanisms include:

- **Heterogeneous connectivity and REST APIs.** Heterogeneous connectivity is used when Live AI Hub needs to query remote databases, SaaS platforms, or other non-local systems through database links, gateways, credentials, or similar connection mechanisms. REST APIs are used when the source is an application, service, or content system that exposes data or actions through endpoints rather than relational tables. Together, these patterns allow Live AI Hub to connect to operational systems, SaaS applications, custom applications, and content repositories while keeping those systems in place. The recommended design is to wrap these connections in curated database objects, tools, or API functions so AI workloads use governed access paths instead of connecting directly to source systems.
- **External tables.** External tables make file-based data in object storage that can be accessed as database objects. In Live AI Hub, this is especially important for multi-cloud data lakes and object stores, including OCI Object Storage, Azure storage, AWS S3, Google Cloud Storage, and S3-compatible environments. External tables allow selected files or lakehouse data to be queried through SQL and made available to AI profiles, agents, or downstream views without copying the full dataset into the database. They are a good fit when the source data is already organized in object storage and the workload only needs governed access to specific paths or objects. For frequent access, performance can be improved using external table caching or by materializing selected results locally.
- **Data sharing** Data sharing is used when datasets are published by another platform or provider through a governed sharing protocol rather than through a conventional database connection. Live AI Hub should specifically support data shared by providers compatible with the **Delta Sharing** protocol. Delta Sharing is useful because it provides an open way to share tables or datasets across platforms, clouds, organizations, and ecosystems without requiring every consumer to build a custom ingestion pipeline. This pattern is well suited for partner data, marketplace data, shared lakehouse datasets, and cross-platform collaboration. Consumed shares should still be treated as governed inputs: expose only the required datasets, document their meaning and freshness, and control AI access through curated objects and AI profiles.
- **Catalog federation** Catalog federation is used when data assets are governed by external catalogs and the goal is to discover, understand, and access those assets through a unified view rather than recreating catalog metadata manually. This is useful in enterprises that already use multiple catalog systems across databases, lakehouses, object stores, and analytics platforms. Catalog federation helps preserve existing ownership, descriptions, classifications, and governance signals while allowing Live AI Hub teams to identify the right data for a use case. It is usually paired with another access mechanism, such as external tables, data sharing, or heterogeneous connectivity, so that discovered assets can be exposed through approved, curated, and semantically enriched objects for AI workloads.

2.3 Performance optimization

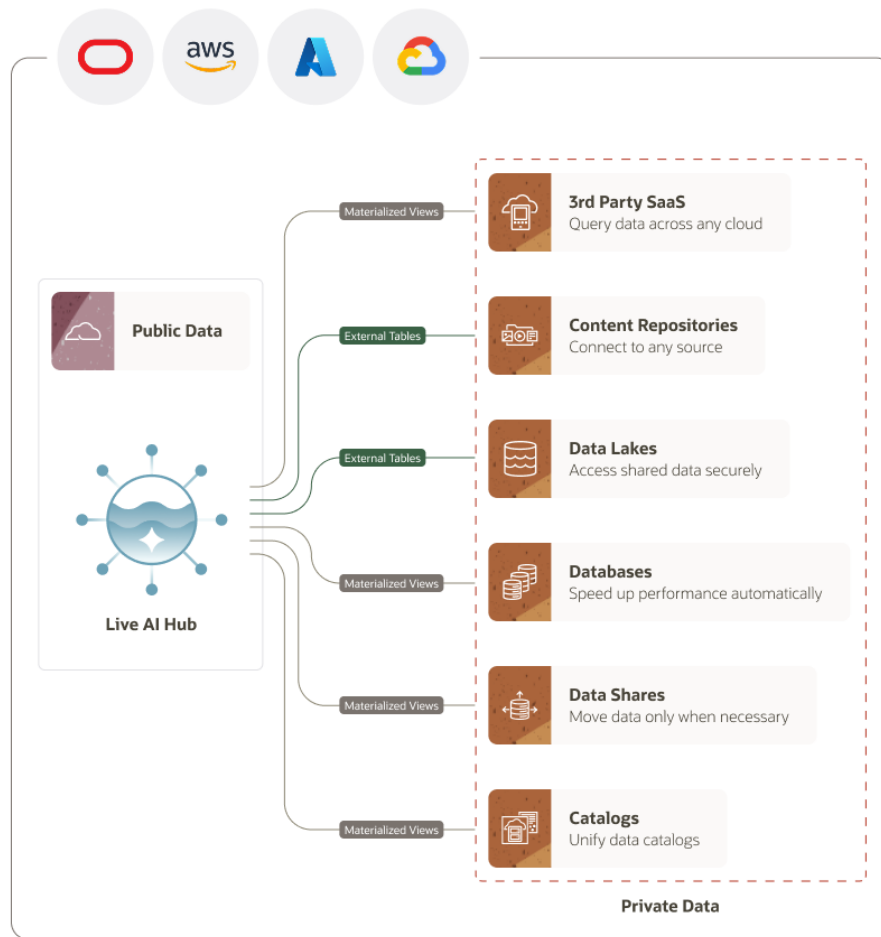


Figure 3. Performance optimization

Live AI Hub prioritizes access to live data, but performance and latency requirements vary by use case. The architecture supports multiple approaches to balance data freshness, cost, responsiveness, and source system load.

Where real-time freshness is required, data can be queried directly from source systems. Where some latency is acceptable, performance can be improved by materializing or caching data locally.

Common approaches include:

- Materialized views.** Materialized views are useful when a Live AI Hub workload repeatedly asks for the same joined, filtered, aggregated, or cross-source data. Instead of forcing every NL2SQL query, agent tool call, or dashboard request to reach across remote systems in real time, the hub can materialize a curated result set close to the AI workload and refresh it on an appropriate schedule. This reduces latency, lowers load on source systems, and makes generated queries more predictable. Materialized views are especially helpful for high-value facts, KPIs, customer or product summaries, and other objects that are expensive to recompute but do not require second-by-second freshness. The design should document the refresh strategy, acceptable staleness, ownership, and dependency on source objects, so users understand when the materialized data is the right access path versus when live federation is required.
- External tables.** External tables provide a way to query file-based data in object storage as database objects, including data stored in OCI Object Storage, Azure storage, AWS S3, Google Cloud Storage, and S3-compatible environments. In a Live AI Hub design, they are often used to expose selected lake, archive, log, product, or data science outputs without first loading the entire dataset into the database. For occasional access, this may be sufficient as a direct federation pattern. For

repeated or latency-sensitive access, the design should consider external table caching, partitioning by path or date, curated views over the external table, or materialized views that persist the most commonly used subset locally. This lets the hub preserve access to data in other clouds and object stores while still giving AI workloads a stable, governed, and performant object to query.

These approaches can improve performance and user experience, reduce load on source systems, and reduce cross-cloud network and egress costs when remote sources are queried repeatedly.

2.4 Metadata and semantics



Figure 4. Metadata and semantics

Once data is accessible, it is typically organized into curated objects to which we can add semantic information. This provides meaning to the objects, making it easier for the LLM to understand and work with the data.

In this document, metadata includes comments, annotations, and schema information attached to those curated objects. Semantics refers to the business meaning captured in that metadata. Together, they help both people and LLMs interpret data consistently.

Where catalog-based federation already provides source metadata, the semantic layer can refine that metadata locally and control what is exposed to AI workloads.

2.5 Security controls

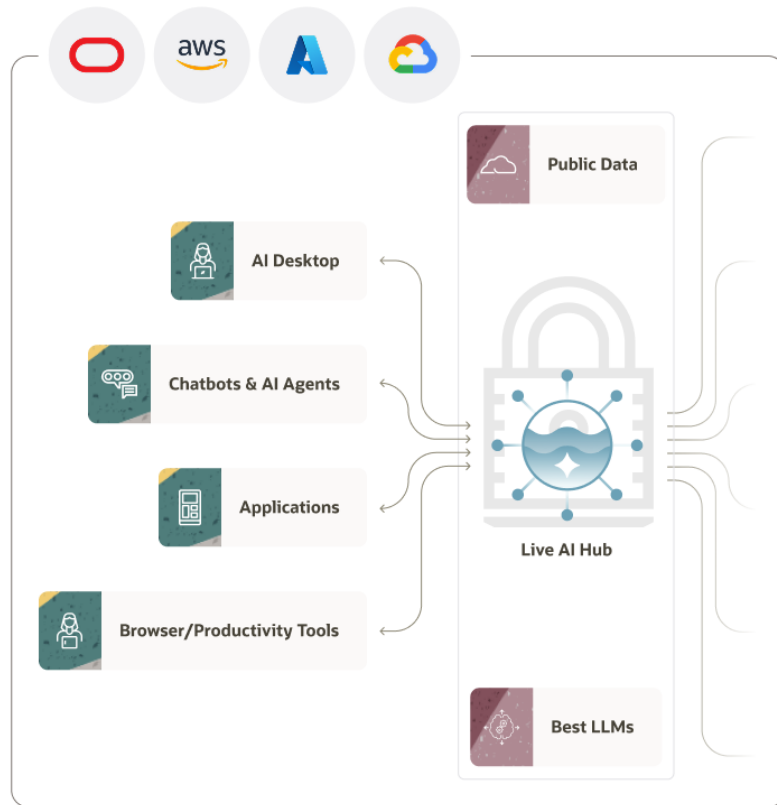


Figure 5. Security controls

Live AI Hub is designed to use the same database security controls that would apply to any enterprise workload, exposing only the data needed for the use case and nothing more.

Security controls commonly used in this architecture include:

- **Least privilege.** Users, applications, AI services, and agents should receive only the privileges required to perform the approved workload. In practice, this means organizing access through roles, granting access to curated views or materialized views rather than broad source schemas, and avoiding direct exposure of underlying tables unless there is a clear need. The same principle should apply to AI runtimes and agent identities: an agent should not be able to query, infer, or act on data beyond the scope of its assigned task. Privilege analysis can be used to validate whether granted privileges are actually needed, and additional controls such as SQL Firewall or Database Vault can help monitor, restrict, or separate privileged activity.
- **Row-level security and fine-grained access.** Row-level security ensures that users and AI agents see only the records they are authorized to access, even when they query the same curated object. In Oracle implementations, Virtual Private Database policies can use application context values, such as a current customer, employee, tenant, region, or business unit, to dynamically filter rows at query time. This is especially important for NL2SQL and agentic workloads because generated SQL and tool calls still execute against governed database objects. The security policy should travel with the object, so access remains consistent whether the request comes from a user interface, a SQL client, a generated query, or an agent tool.
- **Sensitive data protection.** Sensitive data protection limits how personally identifiable information, confidential business data, or regulated attributes are exposed to people, applications, models, and agents. Data redaction can hide or mask selected values in SQL query results, reporting tools, NL2SQL responses, and AI-mediated interactions while still allowing

authorized workflows to operate. This is useful when an AI workload needs to reason over a record but does not need to reveal every sensitive field to the user or model. Sensitive columns should be identified during design, mapped to the relevant policy requirements, and protected through redaction, masking, column filtering, or curated views.

- **Auditing and monitoring.** Live AI Hub workloads should be auditable so teams can understand who accessed which data, through which path, and for what workload. This includes conventional database activity as well as AI-mediated activity such as generated SQL, agent tool calls, API calls, and access to curated views, materialized views, external tables, or vector indexes. Unified Auditing, Data Safe, SQL Firewall events, and workload logs can help security and operations teams detect unusual access patterns, validate policy compliance, and investigate incidents. Auditing should be designed as part of the architecture rather than added later, because production AI workloads need traceability across users, agents, data sources, and actions.

Because federated data is accessed through curated objects in Autonomous AI Database, these controls can be applied consistently regardless of where the underlying data resides. This can also provide an additional layer of protection when source systems have limited controls or inconsistent implementations.

2.6 AI workload layer

Once federation, metadata, performance, and security are in place, the AI workload can be assembled on top of governed data. Live AI Hub supports multiple patterns, including:

- Natural language to SQL (NL2SQL) for structured data questions.
- Agents and agent teams to orchestrate multi-step workflows and optionally take actions through approved tools, such as API calls.
- Retrieval-augmented generation (RAG) with vectors for unstructured content.

Workloads can be invoked through multiple interfaces, including:

- SQL clients
- Application code
- REST APIs
- Other integration patterns like Kafka queues or Streams.
- Agent frameworks that call approved tools exposed by the database, via MCP or A2A interface

The AI workload is created and governed in the Live AI Hub layer, but it can be invoked through different consuming applications and integration patterns. For example, an interactive customer or employee experience may call the workload on demand from a web application or chat interface; a back-office process may invoke the same workload from a scheduled job, data pipeline, queue, or stream; and a developer or analyst may test or operate the workload through SQL, Python, or another database-connected client. In all cases, the client is only the entry point. The workload still executes through approved profiles, tools, agents, database objects, and security controls in the Live AI Hub layer, so the choice of interface should be driven by the surrounding application architecture rather than by a separate AI deployment model.

2.7 Model selection

In multi-agent systems such as customer support, document processing, and content generation, model selection should align with the complexity of the task. Many tasks—such as classification, extraction, or formatting—are relatively simple and benefit more from low latency and low cost than advanced reasoning. Lightweight models are well suited for these high-volume, repeatable operations.

For moderately complex tasks—such as summarization, interpreting user intent, or combining a small number of inputs—mid-tier models provide a balance between performance and cost. These tasks require some contextual understanding but do not need deep reasoning, making mid-tier models effective for the majority of meaningful processing.

Advanced models are best reserved for tasks involving ambiguity, multi-step reasoning, or judgment. Examples include synthesizing insights across sources, resolving conflicting information, or determining next actions in uncertain situations. Although these models have higher cost and latency, they are essential where accuracy and depth are critical.

A practical design pattern is to start with simpler models and escalate when needed. Tasks can initially be handled by faster models, with fallback to more capable models when outputs are incomplete or uncertain. This approach improves efficiency while maintaining quality for complex cases.

Overall, models should be treated as specialized tools, with selection driven by task requirements. Prioritizing speed for simple tasks and accuracy for complex ones enables scalable, cost-effective, and high-performing AI systems. In the design workflow, model selection is part of Step 5 (Create the AI workload) and should be revisited during Step 6 (Operate the workload) as quality, cost, and latency requirements evolve.

2.8 Deployment and locality

Live AI Hub can be deployed in the way that best matches data gravity, user locality, and regulatory constraints. In general, for best performance, run the core engine close to the users and systems it serves, because this reduces network latency and associated network costs. Where possible, place the LLM service near the core engine as well, especially for interactive experiences, to avoid unnecessary round-trip time. Data sources can remain in place. The architecture is designed to access them in place through federation mechanisms and curated objects.

This deployment flexibility supports incremental adoption. Organizations can start with one use case and a small number of sources, then expand to additional sources and workloads as value is demonstrated. The underlying infrastructure scales together with the load.

2.9 Operations, observability and governance

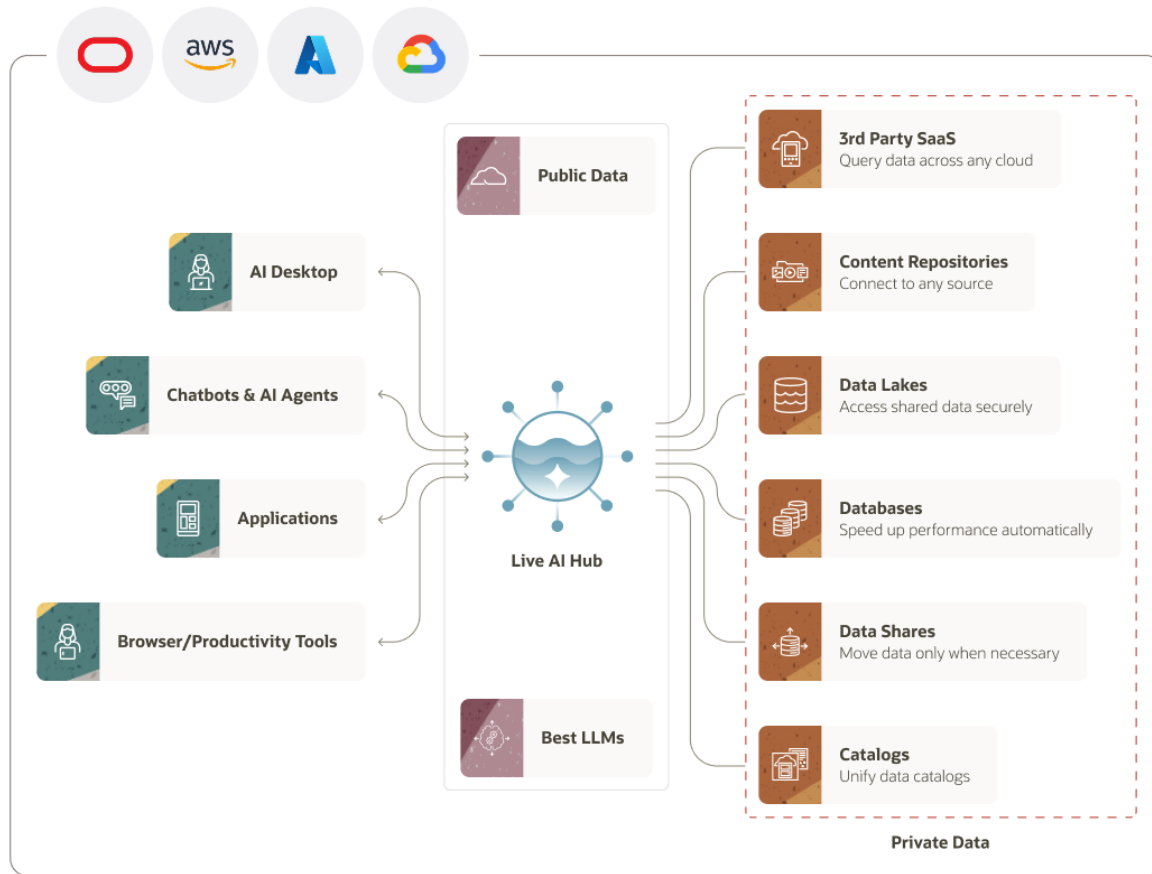


Figure 6. Operations, observability and governance

In this architecture, operations, observability, and governance are designed as part of the AI workload layer, not added as an afterthought. They provide visibility into how workloads execute, which data and tools are used, and how workload behavior changes over time. This applies to both traditional data access paths and AI-mediated access paths such as generated SQL, RAG retrieval, agent tool calls, and downstream actions.

Each workload should produce traceable execution records that capture the user or application identity, entry point, workload or agent profile, model and tool choices, prompt or context metadata, generated queries, retrieved content, tool invocations, source objects accessed, outcomes, and errors. These records make it possible to understand how a result was produced, diagnose unexpected behavior, and verify that the workload operated within approved data and action boundaries.

Operational monitoring should use those records to track latency, error rates, execution frequency, source-system impact, tool usage, retrieval quality, cost drivers, and policy exceptions. Governance should define retention, access to logs, escalation paths, and review processes for new data sources, tools, prompts, models, and agent behaviors. This creates a continuous operating model: observe workload behavior, investigate exceptions, tune metadata and security controls, adjust performance strategies, and improve the AI workload as production usage evolves.

3 Design Guidance

This section describes a recommended, repeatable workflow for designing a Live AI Hub implementation. It applies to the architectural concepts introduced in the [Architecture](#) section and turns them into a sequence of design decisions, validations, and controls.

The steps are ordered deliberately. Each step establishes choices that improve the reliability, performance, semantic quality, security, and operability of the steps that follow. The [Implementation](#) section then maps each step to the relevant Oracle documentation while the [Tutorial](#) section includes source code snippets and links to a complete example implementation.

3.1 Step 1 — Federate data

Goal:

Make the required data reachable where it already lives, so it can be queried in place across hybrid and multi-cloud sources without unnecessary data movement.

Recommended design actions:

The federation mechanism should be selected based on the source technology and how it is exposed.

Credentials should be created and managed for each source.

End-to-end access should be validated by running representative queries against the required objects.

The right set of federated objects should be selected deliberately, based on the use case, the expected AI workload, and the systems of record that can answer the required business questions. There are two common approaches. For a program that already expects several use cases to reuse different parts of the same data landscape, it may be worth onboarding and documenting a broad set of enterprise data up front. This requires more initial effort, but it can reduce duplication later. For a single high-value use case, the faster approach is to start with only the sources and objects needed to answer representative questions then expand the federated scope as new questions, agents, or workflows are added. This makes federation both practical and extensible: start narrow when speed is important, start broader when reuse across multiple use cases justifies the investment, and continuously manage the exposed object set as requirements evolve.

Why it matters:

Federation establishes the foundation for consistent discovery and access to live enterprise data. If the workload cannot reliably and securely reach the required sources, the rest of the design will be unstable.

3.2 Step 2 — Augment performance

Goal:

Performance and latency should meet business needs. “Live” access is only valuable if it returns results within the time required by users and downstream workflows

Recommended design actions:

Freshness requirements should be defined per dataset.

Choose between reading live and accelerating access with caching or materialization.

- Where the freshest data is required, data should be read live.
- If some latency is acceptable, materialize or cache data and refresh it

Why it matters:

Performance is one of the main adoption drivers for AI-assisted experiences. Caching and materialization can also reduce load on source systems and reduce cross-cloud egress costs when remote sources are repeatedly queried

3.3 Step 3 — Augment metadata

Goal:

Establish a semantic layer that makes data understandable to both people and models. This includes curated objects together with the metadata and semantics that describe business meaning.

Recommended design actions:

Source and data semantics should be determined and documented including definitions, intent, and usage.

Table, view, and column comments and/or annotations should be added so the model has the context needed to interpret data correctly.

Semantic consistency should be maintained across curated objects. For example, key measures and statuses should be consistent across views and materialized views.

Why it matters:

Strong semantics typically improves NL2SQL accuracy and reduces ambiguity and hallucinations by providing clearer grounding signals to the model. It also improves determinism in generated SQL by reducing misinterpretation of joins, filters, and business metrics.

3.4 Step 4 — Augment security

Goal:

The workload should be secure and policy-aligned before enabling AI experiences. AI and agents must operate under the same enterprise guardrails as human users.

Recommended design actions:

Least privilege should be enforced for users and for any identities used by AI services and agents. A practical rule is to grant access to the data the AI workload needs for the use case, and no more.

Access to curated objects used by AI, such as views, materialized views, and external tables, should be controlled.

Sensitive data should be protected through the following controls.

Personally identifiable information (PII) should be redacted where appropriate so sensitive values are not exposed to users, tools, or models.

Row-level security should be enforced when needed, for example with VPD, so each user or agent only sees authorized records.

Auditing and security monitoring should be enabled for AI-mediated access, aligned to compliance and investigation requirements.

Why it matters:

This step is the trust gate. It ensures AI outputs are accurate and safe, and that access is auditable in production contexts.

3.5 Step 5 — Create the AI workload

Goal:

The user-facing and/or system-facing AI capabilities should be assembled using the federated, optimized, semantically enriched, and secured data foundation.

Recommended design actions:

AI profiles should be created that define model configuration and scope, including which curated objects should be used and whether comments and annotations are provided to the model.

The right model should be selected for each workload component based on task complexity, latency, cost, and reasoning depth.

Workload placement should be determined for the core engine and LLM service based on user locality, data source distribution, latency, and regulatory constraints.

AI patterns should be implemented depending on the use case:

- NL2SQL for structured data exploration and operational Q&A
- RAG and vectors for grounded answers over unstructured content
- Agents and agent teams for multi-step reasoning and orchestration, including tool invocation such as SQL, retrieval, and API calls

The invocation model should be selected based on the consuming application:

- Interactive, on-demand execution for customer or employee experiences
- Batch or scheduled execution for back-office workflows
- Invocation via SQL, Python, or APIs, depending on the application architecture

Why it matters:

This is where the business experience is delivered, but it should come after federation, performance, semantics, and security are in place. These earlier steps make the workload faster, more reliable, and safer to operate at scale

3.6 Step 6 — Operate the workload

Goal:

The Live AI Hub workload should remain reliable, observable, secure, and effective as usage grows and data, models, and business requirements change.

Recommended design actions:

- Workload behavior and performance should be monitored, including agent runtime characteristics such as duration, tool usage patterns, prompts used and other retained trace data.
- Security and audit signals should be reviewed to confirm the workload continues to operate within policy and investigate unexpected access patterns.
- The semantic layer should be maintained. Update comments and annotations when schemas change or when new curated objects are introduced.
- Caching and refresh schedules should be reassessed as data freshness requirements evolve and query patterns change.
- Model selection should be reevaluated over time. If the selected model does not behave as expected for a given task, consider adjusting the profile configuration or using a different model for that task.
- Reevaluate deployment locality as workload scope expands, source distribution changes, or regulatory requirements evolve.

Why it matters:

Live AI Hub workloads are production systems. Ongoing monitoring and periodic tuning helps maintain performance and data freshness, preserves governance and auditability, and keeps AI outcomes accurate and useful as the environment evolves.

4 Implementation Reference

This section maps each design step to Oracle documentation relevant to implementing Live AI Hub. The references are organized by workflow step so teams can move from design decisions to concrete implementation guidance. Model selection is addressed as part of AI workload design, while deployment and locality should be evaluated alongside federation, workload design, and operations. The corresponding sections in the [Tutorial](#) section show code snippets and links to source code files implementing these steps.

4.1 Federate data

These references show how to connect Autonomous AI Database to enterprise data sources for in-place access.

[Create Database Links from Autonomous AI Database to Another Autonomous AI Database](#)

Enables SQL-based access between Autonomous AI Databases across private or public endpoints, allowing remote Oracle data to be queried within a unified workload.

[Create Database Links to an Oracle Database that is not an Autonomous AI Database](#)

Provides connectivity to external Oracle databases, enabling federated queries across Oracle-managed environments.

[Create Database Links to Non-Oracle Databases with Oracle-Managed Heterogeneous Connectivity](#)

Enables access to non-Oracle databases through managed connectivity, allowing heterogeneous data sources to be queried through a unified SQL interface.

[Query External Data with Autonomous AI Database](#)

Describes tools and packages used to access and validate external data sources across different storage systems.

[Query data in the Autonomous AI Database catalog](#)

Enables discovery and exploration of data available within and connected to Autonomous AI Database through a unified catalog interface.

4.2 Augment performance

These references show how to improve latency, reduce repeated remote reads, and balance freshness, performance, and cost.

[External Table Cache](#)

Enables caching of data from object storage or data lakes to reduce repeated remote reads and improve query latency.

[Materialized Views \(Autonomous Database\)](#)

Stores query results locally with scheduled refresh to improve performance and reduce load on remote or federated data sources.

[Materialized Views — Fast Refresh \(Autonomous Database to Autonomous Database\)](#)

Supports incremental refresh between compatible Oracle databases, improving efficiency for near-real-time access patterns.

[Materialized Views — General Concepts](#)

Provides guidance on balancing data freshness and performance across different workloads and source types.

4.3 Augment metadata

This section links to documentation showing how to build the semantic layer for Live AI Hub by creating curated objects and enriching them with annotations and comments so both users and AI models can interpret business meaning correctly.

[Annotations](#)

Enables structured metadata to be added to tables and columns, providing machine-readable context for improved AI interpretation.

[Comments](#)

Provides descriptive text for tables and columns, supporting understanding of business meaning and usage.

[Using metadata with Select AI](#)

Shows how metadata is used by Select AI to improve NL2SQL accuracy and grounding in enterprise data.

4.4 Augment security

These references show how to enforce least privilege, row-level security, sensitive-data protection, and auditing for AI workloads.

[Oracle Database Security Guide](#)

Provides an overview of database security capabilities, including authentication, authorization, and data protection mechanisms.

[Privileges and Role-Based Access Control \(RBAC\)](#)

Enables least-privilege access by controlling which users and roles can access specific database objects.

[Virtual Private Database \(VPD\)](#)

Enforces row-level security by restricting data visibility based on user or session context.

[Using security with Select AI](#)

Describes how database security controls are applied to AI workloads, including NL2SQL, RAG, and agent-based execution.

[Oracle Data Safe](#)

Provides centralized auditing, activity monitoring, and sensitive data discovery across Autonomous Database environments.

[Using Data Safe with Autonomous Database](#)

Shows how to enable auditing and monitoring for database activity, including AI-mediated access.

4.5 Create the AI workload

These references show how to configure NL2SQL, RAG, vectors, and agents on top of governed, federated data. Model selection should be addressed as part of this step.

[Select AI Overview](#)

Introduces database-integrated AI capabilities for natural language querying, retrieval-augmented generation, and agent-based execution.

[Retrieval-Augmented Generation \(RAG\) with Select AI](#)

Enables grounded responses over unstructured content using vector search and document retrieval.

[Select AI Use Cases](#)

Provides examples of applying NL2SQL, RAG, and agent-based workflows to enterprise scenarios.

[Select AI Agent](#)

Enables orchestration of multi-step workflows using agents that invoke tools such as SQL, retrieval, and APIs.

[Oracle AI Vector Search](#)

Provides vector storage, indexing, and similarity search to support retrieval-based AI patterns.

[Oracle AI Agent Factory](#)

Provides a framework for building and managing AI agents with integrated orchestration and tooling.

4.6 Operate the workload

These references show how to observe, audit, and analyze Live AI Hub workloads in production.

[Agent History and Execution Views](#)

Provides visibility into agent execution, including prompts, tool usage, and workflow steps.

[Audit Autonomous Database Activity](#)

Enables auditing of database activity to monitor data access and operations across AI and non-AI workloads.

5 Tutorial — Worked Example using the Retail Fashion Advisor

This section applies the Live AI Hub workflow introduced in the [Design Guidance](#) section and the [Implementation Reference](#) section to the retail Fashion Advisor example. It shows how a retail assistant can combine live customer, order, product, and policy data with selected external context to deliver personalized recommendations, grounded responses, and guided actions during an active shopping session.

This section is written for implementors such as DBAs and developers who are comfortable with SQL, scripts, database objects, and runtime settings. It points to a [GitHub repository](#) containing all the code for the Fashion Advisor. It highlights relevant Terraform, SQL, shell, and Python assets, identifies where to find the supporting code, and shows how the same patterns can be applied in other Live AI Hub implementations.

5.1 Scenario and success criteria

The retail scenario introduced in the [Overview](#) is a customer-facing Fashion Advisor that is embedded in the retailer's digital shopping experience. The objective is not simply to add a chatbot to the website. The objective is to use live enterprise data and approved business content to help customers discover products, get accurate answers, and complete simple shopping actions while they are actively engaged. In this example, the assistant combines customer profile data, customer analytics, order and commerce data, product and policy content, and selected external context such as weather.

The business value comes from making each customer interaction more relevant and more actionable. Instead of relying on generic recommendations or delayed data copies, the retailer can tailor guidance using current customer context, preferences, history, and analytics insights. The same experience can also answer product, shipping, and return questions using grounded content, and can continue from advice to action through an approved workflow such as adding an item to the shopping cart. This helps the retailer improve digital engagement, increase the likelihood of conversion, strengthen customer satisfaction and loyalty, and get more value from existing systems of record without requiring a separate platform migration.

From the customer's perspective, the experience is more personal and involves less friction. Rather than navigating static menus, filters, and disconnected help pages, the customer can ask for recommendations in natural language, refine the request through conversation, and receive responses based on current data and curated content. The result is a smoother path from discovery to purchase, with answers that are better aligned to the customer's situation at the moment of interaction.

For this worked example, success means that the implementation can:

- combine live data from multiple sources within a single customer interaction;
- deliver personalized, context-aware recommendations;
- provide grounded answers to product and policy questions;
- support a simple approved action in the shopping flow, such as add to cart; and
- do so within the governance, security, and observability model described earlier in this document.

5.2 Architecture mapping

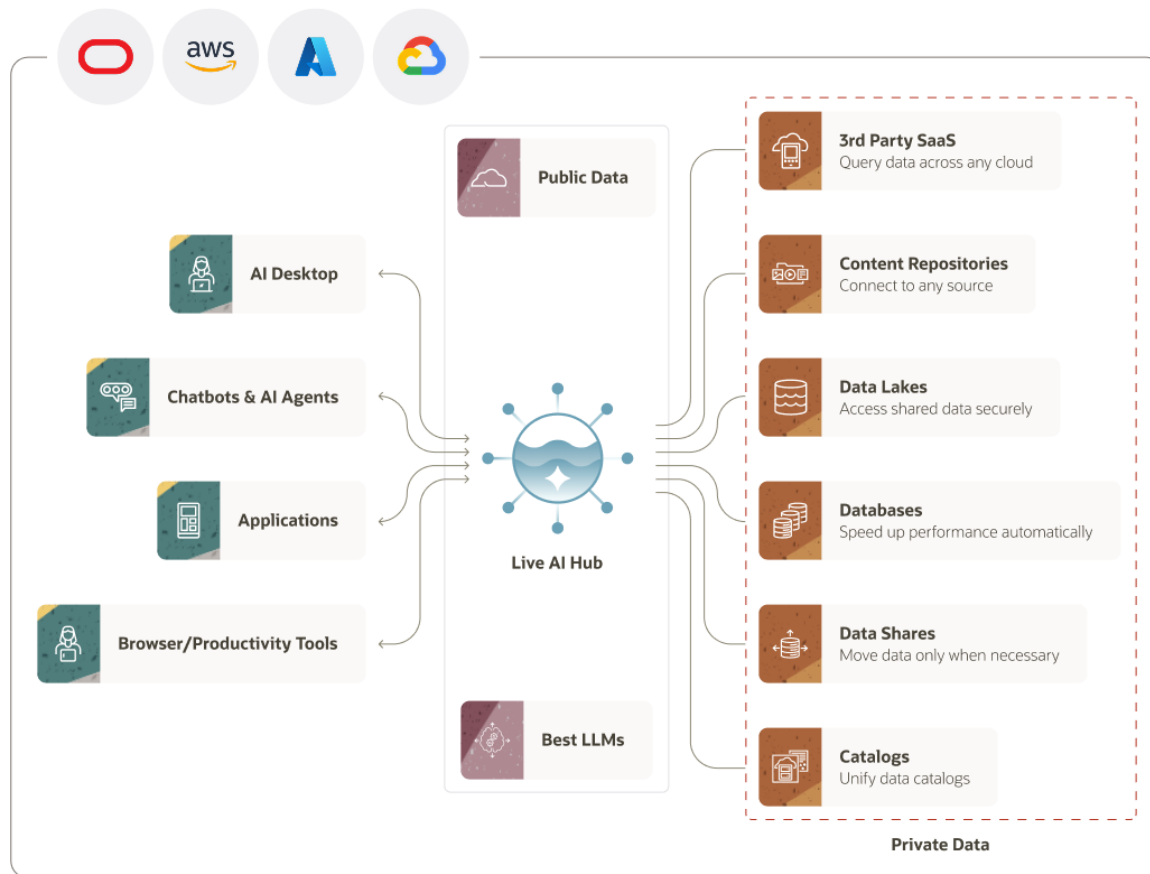


Figure 7. Live AI Hub architecture

The architecture mapping diagram above maps the logical architecture to the retail implementation. Autonomous AI Lakehouse is the central control point for the AI workload. In this worked example, it serves as the primary control plane for semantic access, vector search, and agent execution, while Autonomous AI Transaction Processing (ATP) remains the live commerce store for customer and order data. External customer systems, such as MongoDB for profile data and Databricks for analytics, are accessed as live sources. Unstructured content, including fashion articles and product-related content, is stored in Object Storage and indexed for retrieval. The implementation also defines a separate FAQ-oriented RAG path; deployments should ensure any FAQ or policy documents are uploaded to the configured Object Storage location before enabling that path. Selected public context, such as weather, is accessed through an API. OCI Generative AI provides the models used for NL2SQL, RAG, and agent reasoning.

To keep the scenario concrete, this section describes the customer profile and analytics sources as MongoDB and Databricks. The repository also includes deployment options that simplify some of these integrations for a runnable demo environment, but the architectural pattern remains the same: live structured and unstructured sources are reached through a governed database-centered layer, not by connecting the model directly to each system.

This is the key implementation mapping from the logical architecture. Data flows from source systems into curated database objects, vector indexes, and approved tools inside the Autonomous AI Database layer. That layer is where connections, credentials, semantic annotations, security controls, and observability are applied. In practice, the retail example uses database links and other source-specific connectors for structured federation, Object Storage access for document content, and API calls for selected external services and downstream actions.

The client entry point is the retail web application, which in the reference deployment runs as a Flask application on a Compute VM. A signed-in customer interacts with the Fashion Advisor through that application. The request is handled by the AI workload in the database layer, which retrieves the required live context, invokes the appropriate model and tools, and returns either a grounded response or an approved action outcome. For transactional actions such as add to cart, the flow continues through a commerce API exposed by API Gateway.

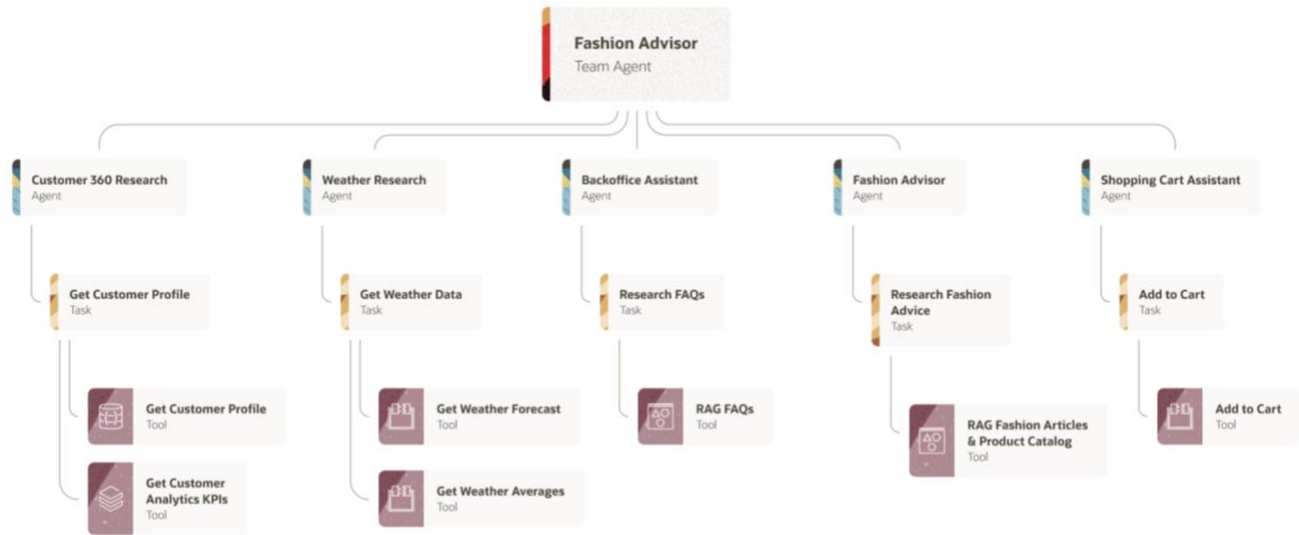


Figure 8. Fashion advisor agents and tools

The diagram above shows how the business outcome is decomposed into a team of specialized agents. The user-facing Fashion Advisor Team Agent orchestrates smaller agents with clear responsibilities, and each of those agents performs tasks using one or more tools.

- The Customer 360 Research Agent retrieves customer profile and customer analytics context.
- The Weather Research Agent retrieves historical weather averages when location or travel context matters.
- The Backoffice Assistant Agent is intended to answer operational questions such as shipping, returns, pricing, and policy by using the FAQ-oriented RAG path when FAQ or policy documents are available.
- The Fashion Advisor Agent uses fashion articles and product-related content to generate recommendations.
- The Shopping Cart Assistant Agent performs the approved downstream action to add an item to the cart.

This decomposition is important because it maps the logical AI workload layer to an implementation that is easier to govern, test, and extend. Each agent has a narrow responsibility, explicit tasks, and defined tools tied to authoritative data sources. That makes the overall workload more predictable and easier to evolve than a single monolithic prompt. The following sections map these architectural elements to the repository assets and walk through how the retail example is built using the same six-step workflow introduced earlier in this document.

5.3 Build steps

This section applies the six-step Live AI Hub workflow to the Fashion Advisor implementation. Each subsection points to corresponding repository assets, linking each implementation topic to code, scripts, and configuration used in the example.

5.3.1 Step 1 — Federate data

The first build step is to connect the five data sources used by the Fashion Advisor example. In this implementation, Autonomous AI Lakehouse acts as the control point and federates each source so the AI workload can use live business data and approved content without requiring a full data consolidation step.

MongoDB

MongoDB provides the customer profile data used to personalize the interaction, such as preferences, style information, sizes, and related profile attributes. This source gives the Fashion Advisor customer-specific context for recommendations.

```
begin
  dbms_cloud.create_credential(
    credential_name => 'mongodb_atlas_cluster_co',
    username => '${mongodb_username}',
    password => '${mongodb_password}'
  );
end;
/

begin
  dbms_cloud_admin.create_database_link(
    db_link_name => 'mongodb_atlas_link_co',
    hostname => '${mongodb_hostname}',
    port => '27017',
    service_name => '${mongodb_db_name}',
    credential_name => 'mongodb_atlas_cluster_co',
    gateway_params => json_object('db_type' value 'mongodb')
  );
end;
/

create materialized view cohub.customer_profile_mv
refresh complete
start with sysdate
next sysdate + 1/4
as
select * from customer_profile@mongodb_atlas_link_co;
```

Follow this [link](#) and search for “setup and connect to the mongodb instance that holds customer profiles”

Autonomous AI Transaction Processing

Autonomous Transaction Processing (ATP) provides the live retail transaction data, including customer, order, and other commerce records. This source gives the Fashion Advisor current operational context from the system of record.

```
BEGIN
  EXECUTE IMMEDIATE 'CREATE OR REPLACE DIRECTORY atp_dblink_wallet_dir AS ''DATA_PUMP_DIR''';
END;
/

begin
  dbms_cloud.get_object(
    credential_name => 'oci',
    object_uri => 'https://objectstorage.${objectstorage_region}.oraclecloud.com/n/${objectstorage_namespace}
    /b/${atp_creds_bucket_name}/o/cwallet.sso',
    directory_name => 'atp_dblink_wallet_dir');
end;
/

begin
  dbms_cloud_admin.create_database_link(
    db_link_name => 'atp_co_link',
    hostname => '${atp_hostname}',
    port => '${atp_port}',
    service_name => '${atp_service_name}',
    ssl_server_cert_dn => '${atp_ssl_server_cert_dn}',
    credential_name => 'atp_co',
    directory_name => 'atp_dblink_wallet_dir');
end;
/
```

```
create view customers_v as select * from customers@atp_co_link;
create view orders_v as select * from orders@atp_co_link;
create view products_v as
select product_id, product_name, unit_price
from products@atp_co_link;
```

Follow this [link](#) and search for “setup and connect via db link to the atp instance”

Customer analytics source (simulation of Databricks)

Customer analytics provide derived context such as customer segment, churn risk, lifetime value, order frequency, and loyalty indicators. In the reference deployment, this Databricks-style analytics source is simulated with data in Autonomous AI Transaction Processing and exposed to Autonomous AI Lakehouse through the `customer_analytics_kpi_mv` object. The same pattern can be adapted to a real external analytics platform.

```
create or replace view customer_analytics_kpi_mv as
select
customer_id,
full_name,
email,
loyalty_tier,
loyalty_points,
total_orders,
lifetime_value,
average_order_value,
customer_segment,
preferred_style,
city,
country
from customer_analytics_kpi@atp_co_link;
```

Follow this [link](#) and search for “create or replace view customer_analytics_kpi_mv”

OCI Object Storage

Object Storage provides the unstructured content used by the retail assistant, including fashion articles and product-related content. The SQL template also defines a separate FAQ vector index, which can support FAQ or policy retrieval when matching documents are uploaded to the configured Object Storage bucket.

```
begin
  dbms_cloud_ai.create_profile(
    profile_name => 'select_ai_rag',
    attributes => '{"provider": "oci",
                  "credential_name": "OCI",
                  "region": "${genai_embedding_region}",
                  "vector_index_name": "hub_vector_index_fashion_articles",
                  "embedding_model": "${select_ai_embedding_model}",
                  "model": "${select_ai_model_rag}"}'
  );
end;
/

begin
  dbms_cloud_ai.create_vector_index(
    index_name => 'hub_vector_index_fashion_articles',
    attributes => '{"vector_db_provider": "oracle",
                  "location": "https://objectstorage.${objectstorage_region}.oraclecloud.com/n/${objectstorage_name}/b/${files_bucket_name}/o/fashion-articles*.md",
                  "object_storage_credential_name": "OCI",
                  "profile_name": "select_ai_rag"}'
  );
end;
/

begin
  dbms_cloud_ai.create_vector_index(
    index_name => 'hub_vector_index_faqs',
```

```

attributes => '{"vector_db_provider": "oracle",
"location": "https://objectstorage.${objectstorage_region}.oraclecloud.com/n/${objectstorage_name}/${files_bucket_name}/o/faq*.md",
"object_storage_credential_name": "OCI",
"profile_name": "select_ai_rag"}'
);
end;
/

```

Follow this [link](#) and search for “*Setup RAG for fashion articles, FAQs*”. There are 2 `dbms_cloud_ai.create_vector_index` calls in this block.

Weather API

The weather API provides selected external context for recommendations that depend on climate or local conditions. This source helps the Fashion Advisor make suggestions that are more relevant to the customer’s situation.

```

create or replace procedure get_weather_averages(
  p_location in varchar2,
  p_country in varchar2,
  p_start_date in date default add_months(sysdate, -12),
  p_end_date in date default sysdate
) is
  l_geocode_resp dbms_cloud_types.resp;
  l_weather_resp dbms_cloud_types.resp;
begin
  l_geocode_resp := dbms_cloud.send_request(
    uri => 'https://geocoding-api.open-meteo.com/v1/search?' ||
      'name=' || utl_url.escape(p_location) || '&count=1',
    method => 'get');

  l_weather_resp := dbms_cloud.send_request(
    uri => 'https://archive-api.open-meteo.com/v1/archive?' ||
      'latitude=' || l_latitude || '&longitude=' || l_longitude ||
      '&start_date=' || to_char(p_start_date, 'yyyy-mm-dd') ||
      '&end_date=' || to_char(p_end_date, 'yyyy-mm-dd') ||
      '&daily=temperature_2m_mean,wind_speed_10m_max,precipitation_sum&timezone=auto',
    method => 'get');
end get_weather_averages;
/

```

Follow this [link](#) and search for “*Get weather data via REST APIs*”

5.3.2 Step 2 — Augment performance

The second build step is to improve response time without losing the freshness required by the retail scenario. In the Fashion Advisor example, performance is tuned per source rather than with a single rule. Data that benefits from repeated fast access during an interactive session is materialized in Autonomous AI Lakehouse with scheduled refresh. Data that must stay current, or that is already fast to access, is queried live. Unstructured content is optimized through the RAG pipeline instead of relational caching.

Materialized view

Customer profile data analytics data is materialized in Autonomous AI Lakehouse (ADW) with automatic refresh. This reduces repeated remote access during the shopping session and gives the assistant faster access to personalization context.

```

create materialized view cohub.customer_profile_mv
refresh complete
start with sysdate
next sysdate + 1/4
as

```

```
select * from customer_profile@mongodb_atlas_link_co;

-- Databricks-style analytics materialization option in the template:
-- create materialized view if not exists customer_analytics_kpi_mv
-- refresh complete next sysdate + 1/4
-- as select * from customer_analytics_kpi;
```

Follow this [link](#) and search for “*create materialized view*”. There are two sections.

Vector index

Object Storage is optimized through the RAG pipeline rather than a materialized view. The implementation reads the documents, chunks them, creates embeddings, and builds a vector index so retrieval runs against the index instead of the raw files.

```
begin
  dbms_cloud_ai.create_profile(
    profile_name => 'select_ai_rag',
    attributes => '{"provider": "oci",
                  "credential_name": "OCI",
                  "region": "${genai_embedding_region}",
                  "vector_index_name": "hub_vector_index_fashion_articles",
                  "embedding_model": "${select_ai_embedding_model}",
                  "model": "${select_ai_model_rag}}"'
  );
end;
/

begin
  dbms_cloud_ai.create_vector_index(
    index_name => 'hub_vector_index_fashion_articles',
    attributes => '{"vector_db_provider": "oracle",
                  "location": "https://objectstorage.${objectstorage_region}.oraclecloud.com/n/${objectstorage_name}
                        space}/b/${files_bucket_name}/o/fashion-articles*.md",
                  "object_storage_credential_name": "OCI",
                  "profile_name": "select_ai_rag"}'
  );
end;
/

begin
  dbms_cloud_ai.create_vector_index(
    index_name => 'hub_vector_index_faqs',
    attributes => '{"vector_db_provider": "oracle",
                  "location": "https://objectstorage.${objectstorage_region}.oraclecloud.com/n/${objectstorage_name}
                        space}/b/${files_bucket_name}/o/faq*.md",
                  "object_storage_credential_name": "OCI",
                  "profile_name": "select_ai_rag"}'
  );
end;
/
```

Follow this [link](#) and search for “*Setup RAG for fashion articles, FAQs*”

5.3.3 Step 3 — Augment metadata

The third build step is to add business meaning to the curated objects used by the Fashion Advisor. This example uses database comments rather than annotations. The implementation adds table and column comments to the customer profile materialized view and the commerce views exposed through ATP database links. These comments describe the business meaning of the objects and columns so Select AI can interpret them more accurately during NL2SQL and agent execution.

```
comment on materialized view cohuh.customer_profile_mv is
'materialized view providing a consolidated customer profile including personal info, account info, purchase
history, clothing preferences, shipping addresses, and marketing preferences.';

comment on column cohuh.customer_profile_mv.customerid is 'numeric customer identifier (number(10)).';
comment on column cohuh.customer_profile_mv.personalinfo_fullname is 'full name of the customer
(varchar2(31)).';
comment on column cohuh.customer_profile_mv.accountinfo_loyaltytier is 'loyalty program tier of the customer
(varchar2(9)).';
comment on column cohuh.customer_profile_mv.clothingpreferences_preferredstyle is 'preferred fashion style
of the customer (varchar2(31)).';
comment on column cohuh.customer_profile_mv.shippingaddresses_1_city is 'city of the primary shipping
address (varchar2(31)).';
```

Follow this [link](#) and search for “Add metadata to the MongoDB customer profile materialized view”

```
comment on table customers_v is 'stores customer information including identification, email address, and
full name';
comment on table products_v is 'contains product catalog information including names, prices, product
details, and product images';
comment on table orders_v is 'records customer orders with order datetime, status, and associated customer
and store information';

comment on column customers_v.customer_id is 'primary key - unique identifier for each customer
(auto-generated integer)';
comment on column products_v.product_name is 'name of the product (required)';
comment on column orders_v.order_status is 'current status of the order (required, maximum 10 characters);
status are COMPLETE, CANCELLED, IN PROGRESS';
```

Follow this [link](#) and search for “Add metadata to the views on top of ATP tables”

The Select AI profiles are configured to use those comments.

```
dbms_cloud_ai.create_profile(
  profile_name => 'select_ai_hub_nl2sql',
  attributes => '{"provider": "oci",
    "region": "${rag_region}",
    "credential_name": "oci",
    "model": "${select_ai_model_nl2sql}",
    "comments": "true",
    "temperature": 0.0,
    "object_list": [
      {"owner": "" || l_owner || "", "name": "customers_v"},
      {"owner": "" || l_owner || "", "name": "customer_profile_mv"},
      {"owner": "" || l_owner || "", "name": "orders_v"},
      {"owner": "" || l_owner || "", "name": "products_v"}
    ]}');
```

Follow this [link](#) and search for “Setup Select AI profile for NL2SQL”, then note the "comments": "true" profile attribute.

This example does not use annotations. For guidance on annotations, see [Augment metadata](#) in [Implementation Reference](#).

5.3.4 Step 4 — Augment security

The fourth build step is to control the privileges required by the Fashion Advisor implementation. In this example, the deployment creates the cohuh schema and grants the roles and package privileges needed to federate data, use Select AI, create agents and tools, work with vectors, and call approved APIs. This demonstrates the least-privilege setup required for the demo runtime; it does not implement the broader security controls described earlier, such as VPD, data redaction, Data Safe, or SQL Firewall.

```

create user cohuh identified by AaBbCcDdEe123#;

grant connect to cohuh;
grant dwrole to cohuh;
grant resource to cohuh;
grant db_developer_role to cohuh;
grant execute on dbms_cloud_ai to cohuh;
grant execute on dbms_cloud to cohuh;
grant execute on dbms_cloud_admin to cohuh;
grant create database link to cohuh;
grant execute on dbms_cloud_ai_agent to cohuh;
grant create any index to cohuh;
grant execute on dbms_vector to cohuh;
grant execute on dbms_cloud_pipeline to cohuh;

```

Follow this [link](#) and search for “*run as an admin user to grant privileges*”

To see the package privileges used by Select AI, federation, agents, vectors, and RAG, search for “*grant privileges to dbms_cloud_ai, dbms_cloud, dbms_cloud_admin*”.

To see the network permissions used for the approved API path, search for “*set up network ACL to access the flask app REST APIs via the OCI API Gateway*”.

This example does not use the other security mechanisms described in the workflow. For guidance on those mechanisms, see [Augment security in Implementation Reference](#).

5.3.5 Step 5 — Create the AI workload

After federation, performance tuning, metadata, and security are in place, the final build step is to assemble the AI workload that the retail application calls. In the Fashion Advisor implementation, this includes creating Select AI profiles for NL2SQL, agent execution, and RAG; building vector indexes over approved documents; defining tools that expose curated data, retrieval, weather, product catalog, and cart actions; grouping those tools into tasks; and orchestrating specialist agents into the Fashion Advisor team. The code in this section shows how the workload is bound to governed database objects and approved tools rather than allowing the model to connect directly to source systems.

Create the Select AI profiles and choose models

Create the Select AI profiles that the workload uses for NL2SQL and agent execution. The profiles identify the models to use, enable database comments as semantic context, and restrict the workload to the curated objects that the Fashion Advisor is allowed to query.

1.

```

begin
  dbms_cloud_ai.create_profile(
    profile_name => 'select_ai_hub_nl2sql',
    attributes   => '{"provider": "oci",
                    "region": "${rag_region}",
                    "credential_name": "oci",
                    "model": "${select_ai_model_nl2sql}",
                    "oci_apiformat": "GENERIC",
                    "comments": "true",
                    "temperature": 0.0,
                    "object_list": [
                      {"owner": " " || l_owner || " ", "name": "customers_v"},
                      {"owner": " " || l_owner || " ", "name": "customer_profile_mv"},
                      {"owner": " " || l_owner || " ", "name": "orders_v"},
                      {"owner": " " || l_owner || " ", "name": "products_v"}
                    ]}' );
end;
/

begin
  dbms_cloud_ai.create_profile(
    profile_name => 'select_ai_hub',
    attributes   => '{"provider": "oci",
                    "region": "${rag_region}",
                    "credential_name": "oci",
                    "model": "${select_ai_model_agent}"',

```

```

        "comments": "true",
        "object_list": [
          {"owner": " " || l_owner || " ", "name": "customers_v"},
          {"owner": " " || l_owner || " ", "name": "customer_profile_mv"},
          {"owner": " " || l_owner || " ", "name": "orders_v"},
          {"owner": " " || l_owner || " ", "name": "products_v"}
        ]});
end;
/

```

Follow this [link](#) and search for “*Setup Select AI profile for NL2SQL*” and “*Setup Select AI profile for Agents*”

```

variable "select_ai_model_nl2sql" {
  type    = string
  default = "xai.grok-4-1-fast-reasoning"
}

variable "select_ai_model_agent" {
  type    = string
  default = "xai.grok-4-fast-non-reasoning"
}

variable "select_ai_model_rag" {
  type    = string
  default = "meta.llama-4-maverick-17b-128e-instruct-fp8"
}

variable "select_ai_embedding_model" {
  type    = string
  default = "cohere.embed-english-v3.0"
}

```

To see where the model values are configured follow this [link](#) and search for “*select_ai_model_nl2sql*”

Create the RAG pipeline and vector index

Create the RAG profiles and vector indexes that turn approved Object Storage documents into searchable knowledge sources. The implementation creates separate vector-index paths for fashion articles and FAQ-style content, using the configured embedding and generation models. The fashion article path is populated by the repository content; the FAQ path requires matching FAQ Markdown files in Object Storage, such as files named `faq.md` or `faq-retail-policy.md`.

```

begin
  dbms_cloud_ai.create_profile(
    profile_name => 'select_ai_rag',
    attributes => '{"provider": "oci",
                  "credential_name": "OCI",
                  "region": "${genai_embedding_region}",
                  "vector_index_name": "hub_vector_index_fashion_articles",
                  "embedding_model": "${select_ai_embedding_model}",
                  "model": "${select_ai_model_rag}"}'
  );
end;
/

begin
  dbms_cloud_ai.create_vector_index(
    index_name => 'hub_vector_index_fashion_articles',
    attributes => '{"vector_db_provider": "oracle",
                  "location": "https://objectstorage.${objectstorage_region}.oraclecloud.com/n/${objectstorage_name}/b/${files_bucket_name}/o/fashion-articles*.md",
                  "object_storage_credential_name": "OCI",
                  "profile_name": "select_ai_rag"}'
  );
end;
/

begin
  dbms_cloud_ai.create_vector_index(
    index_name => 'hub_vector_index_faqs',

```



```

attributes => '{"vector_db_provider": "oracle",
"location": "https://objectstorage.${objectstorage_region}.oraclecloud.com/n/${objectstorage_name}/${files_bucket_name}/o/faq*.md",
"object_storage_credential_name": "OCI",
"profile_name": "select_ai_rag"}'
);
end;
/

```

Follow this [link](#) and search for “Setup RAG for fashion articles, FAQs”

Create the tools

Define the callable tools that give the agents controlled access to data and actions. In this implementation, the tools retrieve customer profile data, customer analytics, weather averages, product catalog information, RAG results, FAQ answers, and cart updates.

2.

```

dbms_cloud_ai_agent.create_tool(
  tool_name => 'get_customer360_profile_tool',
  attributes => '{"instruction": "Get the customer profile for a given customer.",
"function": "get_customer_profile_json",
"tool_inputs": [{"name": "p_customer_name", "description": "Customer name"}]}' ,
  description => 'Tool to get the customer profile given a customer name'
);

dbms_cloud_ai_agent.create_tool(
  tool_name => 'add_cart_item_tool',
  attributes => '{"instruction": "Use this tool to add an item to the shopping cart. Never ask the user for
backend-only product fields.",
"function": "add_cart_item_fn",
"tool_inputs": [
  {"name": "p_product_id", "description": "Identifier of the product to add to the cart.
Use the SKU value."},
  {"name": "p_sku", "description": "SKU of the product to add to the cart."},
  {"name": "p_quantity", "description": "Quantity to add to the cart."},
  {"name": "p_customer_id", "description": "customer_id of the customer asking to add the
item to the cart."},
  {"name": "p_session_id", "description": "session_id of the current customer session."}
]}'
);

```

Follow this [link](#) and search for “create a custom tool to query customer profile” or “create the add cart item tool”

Create the tasks

Define the tasks that group tools into business capabilities the agents can execute. The implementation creates tasks for customer 360 lookup, weather context, fashion advice research, FAQ research, and adding selected products to the shopping cart.

```

dbms_cloud_ai_agent.create_task(
  task_name => 'customer_360',
  attributes => '{"tools": ["get_customer360_profile_tool"], "enable_human_tool" : "false"}'
);

dbms_cloud_ai_agent.create_task(
  task_name => 'get_weather_averages',
  attributes => '{"tools": ["get_weather_averages_tool"], "enable_human_tool" : "false"}'
);

dbms_cloud_ai_agent.create_task(
  task_name => 'research_fashion_advice',
  attributes => '{"input": "customer_360", "tools": ["rag_tool", "get_product_catalog_tool"]}'
);

dbms_cloud_ai_agent.create_task(
  task_name => 'research_faqs',

```

```

attributes => '{"tools": ["rag_tool_faqs"], "enable_human_tool" : "false"}'
);

dbms_cloud_ai_agent.create_task(
  task_name => 'add_cart_item_task',
  attributes => '{"input": "research_fashion_advice", "tools":
    ["add_cart_item_tool", "get_product_catalog_tool"]}'
);

```

Follow this [link](#) and search for “create a customer 360 task”

Create the agents and team orchestration

Create the specialist agents and assemble them into the Fashion Advisor team. The implementation defines agents for customer context, fashion guidance, weather, back-office questions, and shopping cart actions, then coordinates them through a top-level team agent.

```

dbms_cloud_ai_agent.create_team(
  team_name => 'fashion_advisor_team',
  attributes => '{"agents": [
    {"name": "customer360_research_agent", "task": "customer_360"},
    {"name": "weather_research_agent", "task": "get_weather_averages"},
    {"name": "backoffice_agent", "task": "research_faqs"},
    {"name": "fashion_advisor_agent", "task": "research_fashion_advice"},
    {"name": "shopping_cart_assistant", "task": "add_cart_item_task"}
  ],
  "process": "sequential"}'
);

```

Follow this [link](#) and search for “create the fashion advisor agent team”

Run the workload

Test the Fashion Advisor team by creating a conversation, setting the active team, and invoking the agent with sample prompts. This step shows how the assembled workload is executed after the profiles, tools, tasks, and agents have been created.

```

SELECT DBMS_CLOUD_AI.CREATE_CONVERSATION(
  attributes => '{
    "title": "Fashion Advisor",
    "description": "Focus on providing fashion advice; customer is Sam Rueb",
    "retention_days": 14,
    "conversation_length": 15
  }'
) AS conversation_id FROM dual;

exec dbms_cloud_ai_agent.set_team(team_name => 'fashion_advisor_team');

-- select ai agent "I'm Christopher Allen and I'm looking for advice for a coat. What do you recommend?"
-- select ai agent "Are those suggested coats good enough for the winter on the city I live in?"

```

Follow this [link](#) and search for “*dbms_cloud_ai_agent.set_team*”

5.3.6 Step 6 — Operate the workload

The final build step is to understand how the AI workload can be observed after deployment. This documentation does not require readers to run the Fashion Advisor application. Instead, the Fashion Advisor code is used to show the operational surfaces that apply to a Select AI Agent workload: team execution history, task execution history, tool calls, prompt and response records.

Use the Select AI Agent history views to inspect how an agent team ran, which tasks were executed, which tools were called, how long each step took, and whether any step failed. The Fashion Advisor SQL includes example queries for these views.

```
select team.team_exec_id, team.team_name, team.state,
       team.end_date - team.start_date duration,
       team.conversation_id, team.start_date, team.end_date
from user_ai_agent_team_history team
order by team.end_date desc;

select task.task_name, task.end_date - task.start_date duration, task.*
from user_ai_agent_task_history task
order by end_date desc;

select tool.tool_name, tool.end_date - tool.start_date duration, tool.*
from user_ai_agent_tool_history tool
order by invocation_id desc;
```

Follow this [link](#) and search for “check history views to troubleshoot”

To review prompt and response records for a conversation, inspect the Select AI conversation prompt history. The same code block includes an example query against user_cloud_ai_conversation_prompts.

```
select *
from user_cloud_ai_conversation_prompts
where conversation_id = '4A1718D6-73DB-2571-E063-1A13000A2689';

WITH latest_team AS (
  SELECT team_exec_id, team_name, start_date
  FROM user_ai_agent_team_history
  ORDER BY start_date DESC
  FETCH FIRST 1 ROW ONLY
)
SELECT p.prompt, p.prompt_response
FROM latest_team team
JOIN user_ai_agent_task_history task
  ON team.team_exec_id = task.team_exec_id
LEFT JOIN user_cloud_ai_conversation_prompts p
  ON p.conversation_id = JSON_VALUE(task.conversation_params, '$.conversation_id');
```

Follow this [link](#) and search for “get the conversation prompts for a given conversation”

For more guidance see [Operate the workload](#) in [Implementation Reference](#).

6 Glossary

AI agent

A role-based AI runtime that reasons over governed data and uses tools to complete multi-step work.

AI profile

A configuration that selects the model and controls scope, such as allowed objects and metadata inclusion.

Annotation

Structured metadata added to objects or columns to describe meaning and improve AI understanding.

Autonomous AI Database

The core engine that federates data, enforces security, manages performance, and runs AI features.

Catalog

A metadata inventory that helps users and AI discover, interpret, and govern data assets.

Catalog federation

A way to discover and access data governed by external catalogs through a unified catalog view.

Data gravity

The tendency for applications, services, analytics, and AI workloads to move closer to large or critical datasets because moving the data itself is costly, slow, or difficult to govern.

Data redaction

Hides sensitive values at query time so users and AI can access results without exposing PII.

Data Safe

A security service used to assess posture, discover sensitive data, and audit activity.

Data sharing

Publishing and consuming datasets across platforms using standard sharing protocols.

Database link

A database object that enables SQL access to remote data sources as if they were local objects.

Delta Sharing

An open protocol for sharing data securely across platforms without copying full datasets.

Embedding

A numeric representation of text or other content used for similarity search and retrieval.

External table

A database object that lets you query files in object storage using SQL.

External table cache

A performance feature that caches external table data to reduce repeated remote reads.

Federated data access

Accessing and combining data from multiple sources where it resides, using mechanisms such as database links, external tables, APIs, catalog federation, or data sharing.

Federated query

A single query that accesses and joins data across multiple remote or heterogeneous sources.

Freshness

How current the data must be for a use case, often traded off against performance and cost.

Governance

Policies and controls that ensure data is discoverable, secure, auditable, and used appropriately.

Heterogeneous connectivity

Database connectivity that enables access to non-Oracle systems through managed gateways and links.

Large language model (LLM)

A model used to interpret prompts, generate SQL, produce answers, and support agent reasoning.

Materialized view

A locally stored result of a query, refreshed on a schedule to improve performance and reduce source load.

Metadata

Descriptive information about data objects, such as definitions, owners, and column meanings.

Model selection

Choosing an LLM suited to the task, balancing accuracy, reasoning strength, cost, and latency.

NL2SQL

Natural language to SQL generation that lets users ask questions and have SQL executed on approved objects.

Object Allowlist

The explicit list of tables/views an AI profile can access for NL2SQL or agent tools.

Object Storage

Cloud storage for files and documents that can be queried via external tables or indexed for RAG.

Observability

Monitoring workload behavior, performance, and security signals to support reliable production operations.

Personally identifiable information (PII)

Data that can identify a specific individual, either directly or when combined with other data, such as name, email address, phone number, postal address, or customer identifier.

RAG (Retrieval-Augmented Generation)

A pattern that retrieves relevant documents and uses them to ground model responses.

Refresh schedule

The schedule for updating materialized views, caches, or indexes so performance improvements stay aligned with freshness requirements.

Row-level Security

Restricts which rows a user or agent can see, often driven by session context.

Select AI

Database-integrated capabilities for NL2SQL, RAG, and agent workflows using governed data.

Semantic layer

Business meaning captured in metadata, so people and models interpret data consistently.

Task

A defined unit of agent work that specifies goals, tools allowed, constraints, and expected output.

Tool

An executable capability an agent can invoke, such as SQL, RAG retrieval, or an API wrapper function.

Vector index

A structure that enables fast retrieval of relevant content based on embedding similarity.

Vector search

Searching by semantic similarity using embeddings, often used to power RAG retrieval.

Virtual private database (VPD)

Oracle row-level security that automatically adds predicates to queries based on context.

Zero-ETL approach

A design that minimizes ETL pipelines by federating and querying data in place, with optional caching.

DRAFT

Connect with us

Call **+1.800.ORACLE1** or visit **oracle.com**. Outside North America, find your local office at: **oracle.com/contact**.

 blogs.oracle.com

 facebook.com/oracle

 twitter.com/oracle

Copyright © 2026, Oracle and/or its affiliates. This document is provided for information purposes only, and the contents hereof are subject to change without notice. This document is not warranted to be error-free, nor subject to any other warranties or conditions, whether expressed orally or implied in law, including implied warranties and conditions of merchantability or fitness for a particular purpose. We specifically disclaim any liability with respect to this document, and no contractual obligations are formed either directly or indirectly by this document. This document may not be reproduced or transmitted in any form or by any means, electronic or mechanical, for any purpose, without our prior written permission.

Oracle, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.